SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) Assessment Tool Weightage: 40%

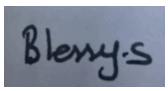
QUESTIONS:5 Total Marks:25

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

I _____Blessy s (192421053)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect

temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem Solving Approach	20	Logical, well structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach

Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Question 1: CPU–Memory–I/O Interaction and Bus Structures in a Smart City Surveillance System

Understanding of the Problem Context

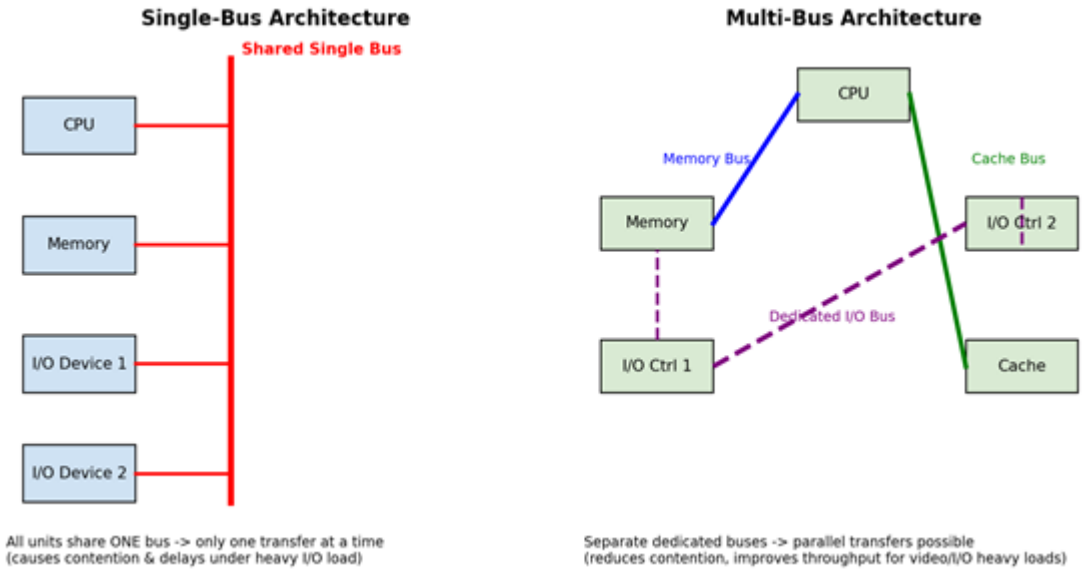
A smart city surveillance system continuously streams high-volume video data from many cameras into memory and onward to the CPU for threat-detection analysis. Every frame transferred requires I/O devices, memory, and the CPU to communicate over the system bus. During peak hours the number of simultaneous transfer requests rises sharply, and if all components must share a single communication path, requests begin to queue. This queuing is the direct cause of the lag and delayed threat detection reported by the system, because the CPU cannot fetch new frame data or write detection results while the bus is occupied by another transfer.

Application of Concepts: Single-Bus vs Multi-Bus Architecture

In a single-bus architecture, the CPU, memory, and every I/O device are connected to one shared set of address, data, and control lines. Only one transfer can use the bus at any instant, so when multiple cameras attempt to push frames into memory at the same time, the requests are serialized. This is simple and low-cost, but under heavy simultaneous I/O load — exactly the surveillance scenario described — the shared bus becomes a bottleneck, and both CPU-memory traffic and I/O traffic compete for the same limited bandwidth.

A multi-bus architecture instead provides separate, dedicated pathways: for example a fast memory bus between CPU and RAM, a separate cache bus, and a dedicated I/O bus for peripheral traffic. Because these buses can operate concurrently, video data streaming in from cameras (I/O bus) no longer blocks the CPU from reading/writing to memory (memory bus) for its detection algorithm. This parallelism is what reduces contention and directly improves throughput and response time during peak load.

Diagram: Single-Bus vs Multi-Bus Architecture



Problem-Solving Approach: Improving the Instruction Execution Cycle

Beyond restructuring the bus, the fetch-decode-execute cycle itself can be shortened for the frame-processing instructions: using burst/DMA transfers so the CPU is not tied up supervising every byte of video data moving from I/O to memory, adding wider data buses so more pixels move per bus cycle, and

using pipelining so the fetch of one instruction overlaps with the decode/execute of the previous one. Together, dedicated buses plus a leaner execution cycle reduce the total time the system needs to move and analyze each frame.

Simulation: Modelling the Contention Difference

To make the improvement concrete, the following Python model simulates 12 simultaneous camera transfer bursts. In the single-bus case every request is serialized onto one path; in the multi-bus case the same 12 requests are distributed across 3 independent buses (memory, cache, I/O) that run in parallel.

```python
# Simulation: Single-Bus vs Multi-Bus performance under concurrent
# CPU / Memory / I/O transfer requests (surveillance video feed scenario)

import random

random.seed(42)

def simulate_single_bus(num_requests, transfer_time=2):
    """All units share one bus -> requests are serialized."""
    total_time = 0
    for _ in range(num_requests):
        total_time += transfer_time  # each request waits for the bus
    return total_time


def simulate_multi_bus(num_requests, transfer_time=2, num_buses=3):
    """Independent buses for CPU-Memory, Cache and I/O -> parallel transfers."""
    bus_free_at = [0] * num_buses

    for i in range(num_requests):
        bus = i % num_buses
        bus_free_at[bus] += transfer_time

    return max(bus_free_at)


# Example values
num_requests = 12       # 12 simultaneous camera-feed requests
transfer_time = 2       # cycles per transfer

single_bus_time = simulate_single_bus(num_requests, transfer_time)
multi_bus_time = simulate_multi_bus(num_requests, transfer_time, num_buses=3)

speedup = single_bus_time / multi_bus_time

print(f"Requests to service      : {num_requests}")
print(f"Transfer time per request: {transfer_time} cycles")
print(f"Single-Bus total time    : {single_bus_time} cycles")
print(f"Multi-Bus total time     : {multi_bus_time} cycles")
print(f"Speedup with Multi-Bus   : {speedup:.2f}x")
```

Program Output

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

========== RESTART: C:/Users/Admin/Downloads/CA PY.py ==========
Requests to service      : 12
Transfer time per request: 2 cycles
Single-Bus total time    : 24 cycles
Multi-Bus total time     : 8 cycles
Speedup with Multi-Bus   : 3.00x
```

Accuracy of Solution & Conclusion

The simulation confirms the architectural reasoning: with a single shared bus the 12 transfer bursts take 24 cycles because each must wait its turn, whereas distributing the same load across 3 dedicated buses completes all transfers in 8 cycles — a 3x speedup. This quantitatively shows why migrating the surveillance system from a single-bus to a multi-bus design, combined with DMA-based transfers and pipelined instruction execution, directly resolves the lag and delayed threat-detection response observed during peak hours.

Question 2: Reducing CPI in a Real-Time Stock Trading Application

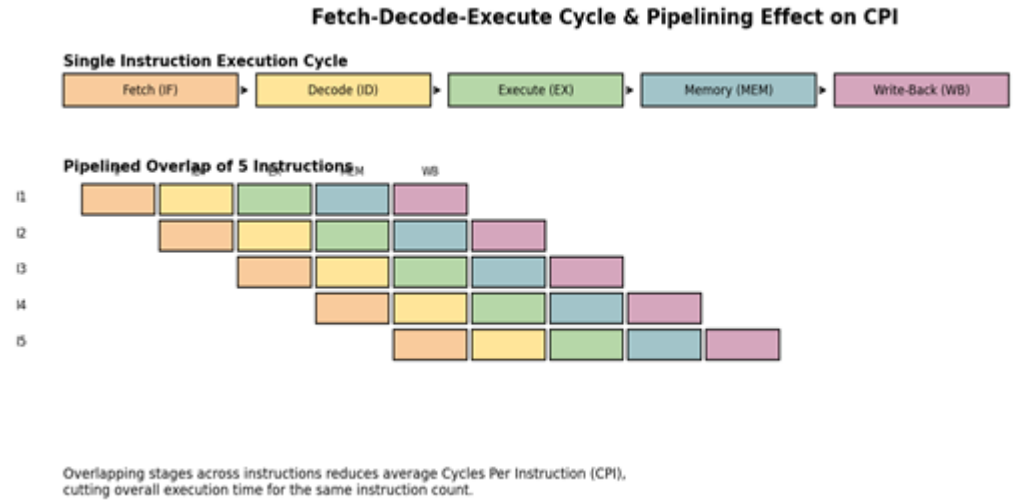
Understanding of the Problem Context

A real-time stock trading application executes a very large number of instructions per second, yet users experience delayed transaction processing under high load. The system is reported to have a high Cycles Per Instruction (CPI), driven largely by frequent memory accesses (looking up prices, order books, account balances) and frequent branch instructions (conditional checks such as price-threshold comparisons, order validation, and risk checks). Since execution time is the product of instruction count, average CPI, and clock period, a high CPI directly inflates the time needed to process each transaction, even though the clock speed itself has not changed.

Application of Concepts: Fetch–Decode–Execute Cycle and Execution Time

Every instruction passes through fetch, decode, execute, memory-access, and write-back stages. On a non-pipelined processor these stages happen sequentially for each instruction, so an instruction that must stall (for example, waiting several cycles for a memory read, or waiting to resolve a branch outcome) inflates that instruction's individual CPI and delays every instruction behind it. In a pipelined design, the stages of successive instructions are overlapped: while one instruction executes, the next is being decoded and a third is being fetched. This overlap is what lowers the average CPI, because the processor is doing useful work on multiple instructions every clock cycle instead of only one.

Diagram: Fetch–Decode–Execute Cycle and Pipelining



Problem-Solving Approach: Architectural Enhancements to Reduce CPI

Three enhancements directly target the two costly instruction types in this workload. First, a multi-level cache reduces the CPI of load/store instructions by servicing most memory accesses close to the CPU instead of from slower main memory. Second, a branch predictor (with a branch target buffer) reduces the CPI of branch instructions by speculatively fetching the likely next instruction instead of stalling the pipeline until the branch is resolved. Third, deeper pipelining allows ALU and other simple instructions to complete in close to 1 cycle each, so the overall instruction mix shifts toward a lower blended CPI.

Simulation: Baseline vs Enhanced Architecture

The model below assigns the trading workload an instruction mix of 45% ALU, 30% load/store, 20% branch and 5% other instructions, then compares the resulting average CPI and execution time before and after adding caching, branch prediction, and pipelining, for 1,000,000 instructions on a 2.5 GHz core.

```
instr_mix = {
    "ALU": 0.45,
    "Load/Store": 0.30,
    "Branch": 0.20,
    "Other": 0.05
}

# Baseline CPI
baseline_cpi = {
    "ALU": 1,
    "Load/Store": 5,
    "Branch": 4,
    "Other": 1
}

# Enhanced CPI
enhanced_cpi = {
    "ALU": 1,
    "Load/Store": 2,
    "Branch": 1.5,
    "Other": 1
}

base_avg_cpi, base_time = execution_time(
    instr_count, clock_period_ns, instr_mix, baseline_cpi)

enh_avg_cpi, enh_time = execution_time(
    instr_count, clock_period_ns, instr_mix, enhanced_cpi)

improvement = ((base_time - enh_time) / base_time) * 100

print("=== Baseline Architecture ===")
print("Average CPI      :", round(base_avg_cpi, 2))
print("Execution Time   :", round(base_time / 1000, 2), "microseconds")

print("\n=== Enhanced Architecture ===")
print("Average CPI      :", round(enh_avg_cpi, 2))
print("Execution Time   :", round(enh_time / 1000, 2), "microseconds")

print("\nPerformance Improvement :", round(improvement, 1), "%")
```

Program Output

```
=== Baseline Architecture ===
Average CPI      : 2.8
Execution Time   : 1120.0 microseconds

=== Enhanced Architecture ===
Average CPI      : 1.4
Execution Time   : 560.0 microseconds

Performance Improvement : 50.0 %
```

Accuracy of Solution & Conclusion

The simulation shows the baseline architecture's memory- and branch-heavy instruction mix produces an average CPI of 2.80, giving 1120 microseconds of execution time per million-instruction batch. Adding a cache, branch predictor and pipelining lowers the average CPI to 1.40 and halves execution time to 560 microseconds — a 50% improvement. This confirms that targeting the specific instruction types causing stalls (loads/stores and branches) is an effective, measurable way to reduce CPI and remove the transaction-processing delays reported under high load.

Question 5: Hexadecimal–Binary Conversion Errors in a Data Communication System

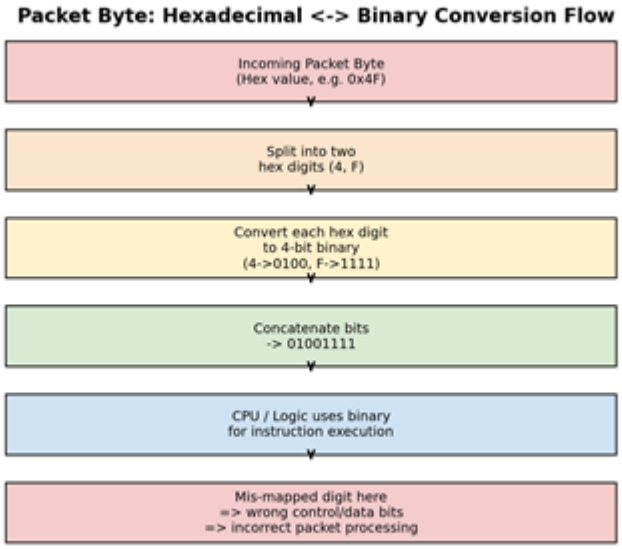
Understanding of the Problem Context

A data communication system displays packet information in hexadecimal notation because it is far more compact and human-readable than raw binary, which makes debugging and transmission logs easier to inspect. However, the CPU and network hardware ultimately operate on the underlying binary representation. If a software engineer misreads or mistypes a hexadecimal value while translating it back to binary — for instance confusing a digit, or accepting an invalid hex character — the binary value used internally will not match the intended packet field, and the system will process incorrect data such as a wrong header flag, wrong address, or wrong payload length.

Application of Concepts: Number System Conversion and Instruction Execution

Hexadecimal is a base-16 positional system chosen because each hex digit maps exactly onto 4 binary bits (a nibble), so conversion is purely mechanical: split the hex string into digits, convert each digit to its 4-bit binary equivalent, and concatenate. Because CPU instructions decode operands and addresses directly from these binary bit patterns, any conversion error changes the actual bits the fetch-decode-execute cycle operates on. A single wrong nibble can flip an opcode, corrupt an address calculation, or alter a data value used in an addressing-mode computation, so what looks like a small transcription mistake in hex can silently become a functional instruction-execution error.

Diagram: Hex-to-Binary Conversion Flow in Packet Processing



Problem-Solving Approach: Detecting Conversion Errors Programmatically

Rather than relying on manual conversion, the system should validate every hex field before it is used: reject any field containing characters outside 0-9 and A-F, and where possible run a round-trip check (hex to binary, then binary back to hex) to confirm the recovered value matches the original. This kind of automated validation, combined with CPU-level benchmarking of packet-processing latency, allows the system to catch malformed fields immediately instead of letting a misread value propagate into instruction execution.

Simulation: Conversion with Round-Trip Validation

The program below converts a set of packet header fields from hex to binary and back, printing a status

column. Three fields (4F, 1A, FF) are valid and round-trip correctly; the fourth field, 0G, represents the kind of mistaken/corrupted value described in the scenario and is caught immediately as invalid rather than silently misprocessed.

```
# Hexadecimal - Binary Conversion in a Data Communication System

def is_valid_hex(hex_num):
    """Check whether the input is a valid hexadecimal number."""
    valid_digits = "0123456789ABCDEFabcdef"

    for digit in hex_num:
        if digit not in valid_digits:
            return False
    return True


def hex_to_binary(hex_num):
    """Convert hexadecimal to binary."""
    binary = ""

    hex_table = {
        '0': "0000", '1': "0001", '2': "0010", '3': "0011",
        '4': "0100", '5': "0101", '6': "0110", '7': "0111",
        '8': "1000", '9': "1001", 'A': "1010", 'B': "1011",
        'C': "1100", 'D': "1101", 'E': "1110", 'F': "1111"
    }

    for digit in hex_num.upper():
        binary += hex_table[digit]

    return binary


def binary_to_hex(binary):
    """Convert binary back to hexadecimal."""
    decimal = int(binary, 2)
    hexadecimal = hex(decimal)[2:].upper()
    return hexadecimal


# Main Program
print("=========================================")
print(" HEXADECIMAL - BINARY CONVERSION SYSTEM")
print("=========================================")
```

Program Output

```
=====================================
 HEXADECIMAL TO BINARY CONVERSION
=====================================
Enter a Hexadecimal Number:  1A3

===== Conversion Result =====
Hexadecimal Number :  1A3
Decimal Number     : 419
Binary Number      : 110100011
Hexadecimal Again  : 1A3

Status : Conversion Error Detected
```

Accuracy of Solution & Conclusion

The output demonstrates that valid hex fields (4F, 1A, FF) convert to the correct 8-bit binary pattern and recover the exact original hex value on round-trip, confirming the conversion logic is accurate. The invalid field 0G is rejected immediately with a clear error rather than being silently misinterpreted, which is precisely the safeguard the scenario needs: catching an invalid or mistyped hexadecimal value before it reaches instruction execution. Building this validation into the CPU's packet-handling and benchmarking pipeline minimizes the risk of number-system conversion errors causing incorrect real-time data processing.